

Метод reverse

Метод `reverse` изменяет порядок элементов в массиве на обратный.

Метод *изменяет исходный массив* (он станет перевернутым) и возвращает также перевернутый массив (можно пользоваться и так, и так).

Синтаксис

```
массив.reverse();
```

Пример

Давайте перевернем массив:

```
let arr = ['a', 'b', 'c'];
```

```
arr.reverse();  
console.log(arr);
```

Результат выполнения кода:

```
['c', 'b', 'a']
```

Пример

А теперь давайте запишем перевернутый массив в новую переменную:

```
let arr = ['a', 'b', 'c'];  
let res = arr.reverse();
```

```
console.log(res);
```

Результат выполнения кода:

```
['c', 'b', 'a']
```

Пример . Применение

Давайте перевернем символы строки в обратном порядке. Для этого разобьем строку в массив с помощью `split` по разделителю " (этот разделитель положит каждый символ строки в отдельный элемент массива), перевернем этот массив с помощью `reverse` и затем сольем перевернутый массив обратно с помощью `join`:

```
let str = '123456789';
```

```
let arr1 = str.split("");  
let arr2 = arr1.reverse();  
let result = arr2.join("");
```

```
console.log(result);
```

Результат выполнения кода:

```
'987654321'
```

Пример . Применение

Упростим решение предыдущей задачи - сольем все команды в цепочку:

```
let str = '123456789';
```

```
let result = str.split("").reverse().join("");  
console.log(result);
```

Результат выполнения кода:

```
'987654321'
```

Метод push

Метод `push` добавляет неограниченное количество элементов в конец массива. При этом исходный массив изменяется, а результатом возвращается новая длина массива.

Синтаксис

массив.`push`(элемент, элемент, элемент...)

Пример

Давайте в конец массива добавим еще два новых элемента:

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.push('1', '2');  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', 'c', 'd', 'e', '1', '2']
```

Пример

Добавим в массив два новых элемента и выведем новую длину массива:

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let length = arr.push('1', '2');
```

```
console.log(length);
```

Результат выполнения кода:

7

Пример . Применение

Заполним массив числами от 1 до 9:

```
let arr = [];  
  
for (let i = 1; i <= 9; i++) {  
    arr.push(i)  
}
```

```
console.log(arr);
```

Результат выполнения кода:

[1, 2, 3, 4, 5, 6, 7, 8, 9]

Метод unshift

Метод `unshift` добавляет неограниченное количество новых элементов в начало массива. При этом исходный массив изменяется, а результатом возвращается новая длина массива.

Синтаксис

```
массив.unshift(элемент, элемент, элемент...);
```

Пример

Давайте в начало массива добавим еще два новых элемента и выведем измененный массив:

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.unshift('1', '2');
```

```
console.log(arr);
```

Результат выполнения кода:

```
['1', '2', 'a', 'b', 'c', 'd', 'e']
```

Пример

Давайте добавим два новых элемента и выведем новую длину массива:

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
let length = arr.unshift('1', '2');
```

```
console.log(length);
```

Результат выполнения кода:

```
7
```

Метод shift

Метод `shift` удаляет первый элемент из массива. При этом исходный массив изменяется, а результатом метода возвращается удаленный элемент.

Синтаксис

```
массив.shift();
```

Пример

Давайте удалим первый элемент из массива:

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.shift();
```

```
console.log(arr);
```

Результат выполнения кода:

```
['b', 'c', 'd', 'e']
```

Пример

Давайте удалим первый элемент из массива и выведем его на экран:

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let elem = arr.shift();
```

```
console.log(elem);
```

Результат выполнения кода:

'a'

Пример . Применение

Пусть дан массив ['1', '2', '3', '4', '5', '6']. Давайте сделаем из него строку '16-25-34'. Для решения задачи используем комбинацию методов `shift`, `pop`, `push` и `join`:

```
let arr = ['1', '2', '3', '4', '5', '6'];  
let result = [];
```

```
while (arr.length > 0) { // массив уменьшается в цикле пока не достигнет нуля  
    let first = arr.shift();  
    let last = arr.pop();  
  
    let str = first + last; // тут будет строка '16', потом '25', потом '34'  
    result.push(str);  
}
```

```
// После цикла в result лежит массив ['16', '25', '34']. Сольем его в строку:  
result = result.join('-');  
console.log(result);
```

Результат выполнения кода:

'16-25-34'

Метод pop

Метод `pop` удаляет последний элемент из массива. При этом исходный массив изменяется, а результатом метода возвращается удаленный элемент.

Синтаксис

массив.`pop()`

Пример

Давайте удалим из массива последний элемент:

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.pop();  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', 'c', 'd']
```

Пример

Давайте выведем последний элемент, который был удален из исходного массива:

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let del = arr.pop();
```

```
console.log(del);
```

Результат выполнения кода:

```
'e'
```

Пример . Применение

Пусть дан массив ['1', '2', '3', '4', '5', '6']. Давайте сделаем из него строку '16-25-34'. Для решения задачи используем комбинацию методов `pop`, `shift`, `push` и `join`:

```
let arr = ['1', '2', '3', '4', '5', '6'];  
let result = [];
```

```
while (arr.length > 0) { // массив уменьшается в цикле пока не достигнет нуля  
  let first = arr.shift();  
  let last = arr.pop();  
  
  let str = first + last; // тут будет строка '16', потом '25', потом '34'  
  result.push(str);  
}
```

```
// После цикла в result лежит массив ['16', '25', '34']. Сольем его в строку:  
result = result.join('-');  
console.log(result);
```

Результат выполнения кода:

'16-25-34'

Метод slice

Метод `slice` вырезает и возвращает указанную часть массива.

Первым параметром указывается номер элемента массива, с которого начинается вырезание, а вторым параметром - номер элемента, на котором закончится вырезание (при этом элемент с этим номером не включится в вырезанную часть).

Второй параметр не является обязательным. Если его не указать - подмассив возьмется с указанного в первом параметре элемента до конца массива.

Второй параметр также может принимать отрицательные значения. В этом случае отсчет элемента, на котором закончится обрезание, начинается с конца массива. Причем, последний элемент имеет номер -1, предпоследний - 2 и так далее.

К примеру, если написать `slice(1, -1)`, то вырезанная часть возьмется с первого элемента включительно по последний не включая его. Если вам нужно включить и последний элемент - нужно просто не задавать второй параметр для `slice`, вот так: `slice(1)`.

Метод не изменяет исходный массив.

Напоминаю, что в массиве нумерация элементов начинается с нуля.

Синтаксис

массив.`slice`(откуда отрезать, [докуда отрезать])

Пример

Давайте вырежем из массива элементы с нулевого по второй не включительно (второй не вырежется):

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let sub = arr.slice(0, 2);
```

```
console.log(sub);
```

Результат выполнения кода:

```
['a', 'b']
```

Пример

Давайте вырежем с первого элемента до конца массива. Для этого второй параметр не задаем:

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let sub = arr.slice(1);
```

```
console.log(sub);
```

Результат выполнения кода:

```
['b', 'c', 'd', 'e']
```

Пример

Давайте вырежем элементы со второго по предпоследний (-1 указывает на последний элемент и он не включится в извлеченную часть):

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let sub = arr.slice(1, -1);
```

```
console.log(sub);
```

Результат выполнения кода:

```
['b', 'c', 'd']
```

Преимущество такого подхода в том, что вырезаться всегда будет часть массива, не включая последний элемент, **независимо** от размера массива.

Метод splice

Метод `splice` удаляет или добавляет элементы в массив. Можно только удалять элементы, только добавлять или делать и то и другое одновременно. Метод очень универсальный и сложный для понимания.

Метод изменяет сам массив и возвращает при этом массив удаленных элементов.

Первым параметром метод принимает номер элемента массива, который нужно удалить. Вторым параметром - сколько элементов массива следует удалить. Если его поставить в 0 - то элементы удалены не будут (только добавлены новые).

Дальше через запятую идут элементы, которые нужно добавить в массив (являются необязательными параметрами). Эти элементы добавятся вместо удаленных элементов массива.

Если удаления не было (когда второй параметр 0) - элементы вставятся в массив начиная с той позиции, которая указана первым параметром метода.

Первый параметр может иметь отрицательное значение. В этом случае отсчет позиции начнется не с начала массива, а с конца. Причем, последний элемент имеет номер -1, предпоследний -2 и так далее.

Синтаксис

массив.[splice](#)(откуда удаляем, сколько элементов удаляем, [вставить элемент], [вставить элемент]...);

Пример

Давайте удалим элементы с первого, 3 штуки (это 'b', 'c' и 'd'):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.splice(1, 3);  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'e']
```

Пример

Давайте выведем массив удаленных элементов:

```
let arr = ['a', 'b', 'c', 'd', 'e'];  
let del = arr.splice(1, 3);
```

```
console.log(del);
```

Результат выполнения кода:

```
['b', 'c', 'd']
```

Пример

Давайте сначала удалим элемент с номером 2, а потом вместо него вставим еще 3 новых элемента ('1', '2' и '3'):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.splice(2, 1, '1', '2', '3');  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', '1', '2', '3', 'd', 'e']
```

Пример

Давайте теперь ничего не будем удалять, но на позицию 2 (это значит перед элементом с номером 2), вставим еще 3 новых элемента ('1', '2' и '3'):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.splice(2, 0, '1', '2', '3');  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', '1', '2', '3', 'c', 'd', 'e']
```

Пример . Отрицательное значение

Давайте удалим предпоследний элемент 'd' (его номер -2):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
```

```
arr.splice(-2, 1);  
console.log(arr);
```

Результат выполнения кода:

```
['a', 'b', 'c', 'e']
```

Метод sort

Метод `sort` производит сортировку массива в лексикографическом порядке.

Также можно указать параметром собственную функцию для сортировки, но это необязательно.

Синтаксис

```
массив.sort([функция])
```

Пример

Давайте посортируем массив:

```
let arr = ["Банан", "Апельсин", "Яблоко", "Манго"];  
console.log(arr.sort());
```

Результат выполнения кода:

```
[Апельсин, Банан, Манго, Яблоко]
```

Функция Object.keys

Функция **Object.keys** позволяет получить ключи объекта в виде массива.

Синтаксис

```
Object.keys(объект);
```

Примеры

Пример

Давайте получим ключи объекта в виде массива:

```
var obj = {a: 1, b: 2, c: 3};  
console.log(Object.keys(obj));
```

Результат выполнения кода:

```
['a', 'b', 'c']
```

Задачи для решения

Работа с concat

Для решения задач данного блока вам понадобятся следующие методы: concat.

- 1 Даны два массива: **[1, 2, 3]** и **[4, 5, 6]**. Объедините их вместе.

Работа с reverse

Для решения задач данного блока вам понадобятся следующие методы: reverse.

- 2 Дан массив **[1, 2, 3]**. Сделайте из него массив **[3, 2, 1]**.

Работа с push, unshift

Для решения задач данного блока вам понадобятся следующие методы: push, unshift.

- 3 Дан массив **[1, 2, 3]**. Добавьте ему в конец элементы **4, 5, 6**.
- 4 Дан массив **[1, 2, 3]**. Добавьте ему в начало элементы **4, 5, 6**.

Работа с shift, pop

Для решения задач данного блока вам понадобятся следующие методы: shift, pop.

- 5 Дан массив **['js', 'css', 'jq']**. Выведите на экран **первый** элемент.
- 6 Дан массив **['js', 'css', 'jq']**. Выведите на экран **последний** элемент.

Работа с slice

Для решения задач данного блока вам понадобятся следующие методы: slice.

- 7 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **slice** запишите в новый массив элементы **[1, 2, 3]**.
- 8 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **slice** запишите в новый массив элементы **[4, 5]**.

Работа с splice

Для решения задач данного блока вам понадобятся следующие методы: splice.

- 9 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **splice** преобразуйте массив в **[1, 4, 5]**.
- 10 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **splice** запишите в новый массив элементы **[2, 3, 4]**.
- 11 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **splice** сделайте из него массив **[1, 2, 3, 'a', 'b', 'c', 4, 5]**.
- 12 Дан массив **[1, 2, 3, 4, 5]**. С помощью метода **splice** сделайте из него массив **[1, 'a', 'b', 2, 3, 4, 'c', 5, 'e']**.

Работа с sort

Для решения задач данного блока вам понадобятся следующие методы: sort.

- 13 Дан массив **[3, 4, 1, 2, 7]**. Отсортируйте его.

Работа с Object.keys

Для решения задач данного блока вам понадобятся следующие методы: Object.keys.

14 Дан объект {js: 'test', jq: 'hello', css: 'world'}. Получите массив его **ключей**.

В следующих задачах вы должны применить комбинацию стандартных функций JavaScript, чтобы решить задачу.

15 Дана строка. Сделайте заглавным первый символ этой строки не используя цикл. Найдите два решения.

16 Дана строка, например, '123456'. Переверните эту строку (сделайте из нее '654321') не используя цикл.

17 Проверьте, что строка начинается на **http://**.

18 Проверьте, что строка заканчивается на **.html**.

